COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S2
DATE PERFORMED:	09/06/2026	**DATE SUBMITTED:**	09/06/2026
NAME:	Juan Paulo C. Lara	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 7.1 SORTING ALGORITHMS: BUBBLE, SELECTION, AND INSERTION SORT

1. PROCEDURE OUTPUT

What procedures did you perform?

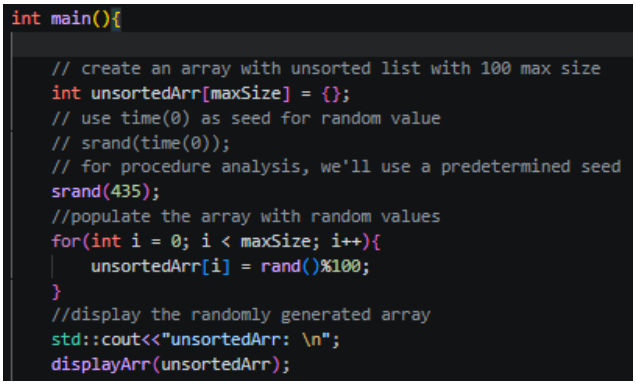
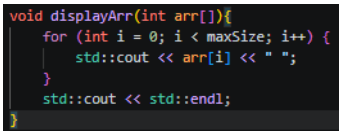
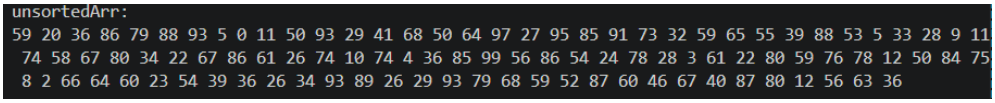
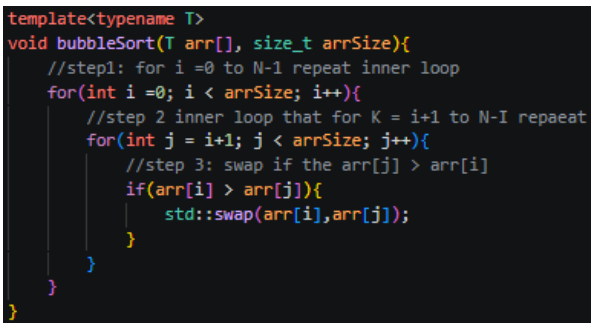
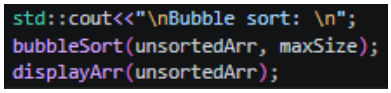
Code + Console	  
Observations	This snippet prints an unsorted array by using the display function to display the randomly generated set of values. To generate these values while remaining reproducible for analysis, a predetermined seed integer number is used for the randomizer function. The first loop writes a pseudorandom number to an element for each iteration, and the display loop reads and displays the stored values.

Table 7-1. Array of Values for Sort Algorithm Testing

Code + Console	 

	unsortedArr: 59 20 36 86 79 88 93 5 0 11 50 93 29 41 68 50 64 97 27 95 85 91 73 32 59 65 55 39 88 53 5 33 28 9 11 74 58 67 80 34 22 67 86 61 26 74 10 74 4 36 85 99 56 86 54 24 78 28 3 61 22 80 59 76 78 12 50 84 75 8 2 66 64 60 23 54 39 36 26 34 93 89 26 29 93 79 68 59 52 87 60 46 67 40 87 80 12 56 63 36 Bubble sort: 0 2 3 4 5 5 8 9 10 11 11 12 12 20 22 22 23 24 26 26 26 27 28 28 29 29 32 33 34 34 36 36 36 36 39 39 40 41 46 50 50 50 52 53 54 54 55 56 56 58 59 59 59 59 60 60 61 61 63 64 64 65 66 67 67 67 68 68 73 7 4 74 74 75 76 78 78 79 79 80 80 80 84 85 85 86 86 86 87 87 88 88 89 91 93 93 93 93 95 97 99
Observations	A bubble sorting algorithm uses a nested for loop and a swapping function (using std::swap) to sort the randomly generated unsorted array. There is a nested loop because the inner loop performs the swapping and the outer loop acts as a repeater of the loop until the condition that the entire list is sorted. Therefore, the average and worst case scenario of this algorithm is $O(n^2)$.

Table 7-2. Bubble Sort Technique

Code + Console	`template<typename T>`
	`//array, current position, size of array`
	`int routineSmallest(T arr[], int K, const int arrSize){`
	`  int position = K;`
	`  T smallestElem = arr[K];`
	
	`  for(int j = K+1; j<arrSize; j++){`
	`  if(arr[j] < smallestElem){`
	`  smallestElem = arr[j];`
	`  position = j;`
	`  }`
	`  }`
	`  return position;`
	`}`
	
	`std::cout<<"\nSelection sort: \n";`
	`selectionSort(unsortedArr, maxSize);`
	`displayArr(unsortedArr);`
	
	`template<typename T>`
	`void selectionSort(T arr[], size_t arrSize){`
	`  int POS, temp, pass;`
	`  for(int i=0; i < arrSize; i++){`
	`  POS = routineSmallest(arr, i, arrSize);`
	`  //swapping:`
	`  temp = arr[i];`
	`  arr[i] = arr[POS];`
	`  arr[POS] = temp;`
	
	`  //for verification and counting of numb`
	`  was done`
	`  pass++;`
	`  }`
	`}`
	
	unsortedArr:
	59 20 36 86 79 88 93 5 0 11 50 93 29 41 68 50 64 97 27 95 85 91 73 32 59 65 55 39 88 53 5 33 28 9 11
	74 58 67 80 34 22 67 86 61 26 74 10 74 4 36 85 99 56 86 54 24 78 28 3 61 22 80 59 76 78 12 50 84 75
	8 2 66 64 60 23 54 39 36 26 34 93 89 26 29 93 79 68 59 52 87 60 46 67 40 87 80 12 56 63 36
	
	Selection sort:
	0 2 3 4 5 5 8 9 10 11 11 12 12 20 22 22 23 24 26 26 26 27 28 28 29 29 32 33 34 34 36 36 36 36 39 39
	40 41 46 50 50 50 52 53 54 54 55 56 56 58 59 59 59 59 60 60 61 61 63 64 64 65 66 67 67 67 68 68 73 7
	4 74 74 75 76 78 78 79 79 80 80 80 84 85 85 86 86 86 87 87 88 88 89 91 93 93 93 93 95 97 99

Observations	This selection sorting algorithm performs its sorting routines by finding the smallest element from the unsorted part of the array, and swapping this value to the first unsorted element. For

	each iteration, it calls a helper function to find the minimum values, and sets a temporary variable to store the swapping values. Since it uses a helper function which has another loop, the best and worst case scenario for this algorithm is $O(n^2)$.

Table 7-3. Selection Sort Algorithm

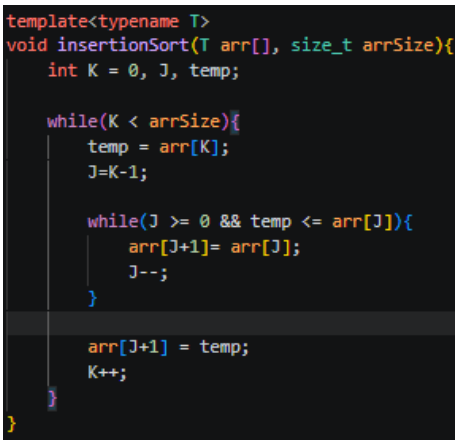
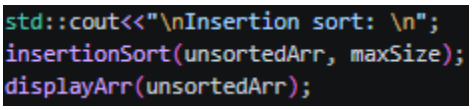
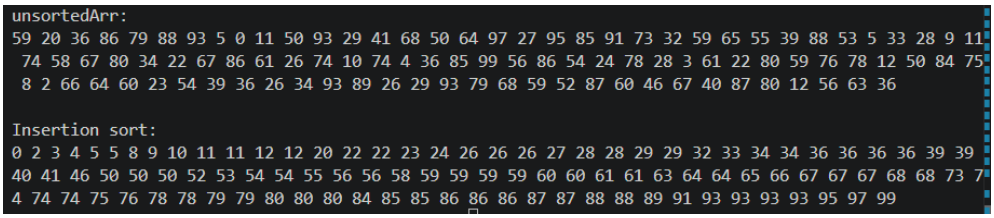
Code + Console	  
Observations	This insertion sort algorithm sequentially sorts elements by starting from the first index, taking the value for comparison, and then shifting and inserting it to a sublist in the correct arrangement. Unlike the previous method, this does not use a helper function to find a certain value involved in the main sorting.Therefore, the best case and worst case scenario for this algorithm is $O(N)$ and $O(N^2)$ respectively.

Table 7-4. Insertion Sort Algorithm

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

FUNCTION MAIN
START

DELCARE unsortedArr[maxSize]
DECLARE candidates

RANDOMIZE value

```
        LOOP i to maxSize
                unsortedArr[i] = RANDOM value (range 1-5) + 1

        PRINT unsortedArr
        CALL displayArr

        PRINT unsortedArr
        CALL selectionSort
        CALL displayArr

        DECLARE votes[5]
        CALL countVotes

        PRINT "Vote Results"

        LOOP i to 5
                PRINT candidates[i], votes[i]

        DECLARE winner

        LOOP i to 5
                IF votes[i] > votes[winner]
                winner = i

        PRINT candidates[winner], votes[winner]

        END

FUNCTION displayArr
        START
        LOOP i to maxSize
                PRINT arr[i]
        PRINT endl
        END

FUNCTION insertionSort
        START
        DECLARE k, J, temp[]
        LOOP k to arrSize
                temp = arr[K]
                J = K-1

                LOOP J >= 0 AND temp <= arr[J]
                arr[J+1]= arr[J]
                DECREMENT J

        arr[J+1] = temp
```

```
        INCREMENT K
        END

FUNCTION countVotes
        START
        LOOP i to 5
                votes[i] = 0
        LOOP i to arrSize
                INCREMENT votes[arr[i] - 1]
        END
```

Table 8-1. Pseudocode of Algorithm.

```
template<typename T>
void insertionSort(T arr[], size_t arrSize){
    int K = 0, J, temp;

    while(K < arrSize){
        temp = arr[K];
        J=K-1;

        while(J >= 0 && temp <= arr[J]){
            arr[J+1]= arr[J];
            J--;
        }

        arr[J+1] = temp;
        K++;
    }
}
```

```
template<typename T>
void countVotes(T arr[], size_t arrSize, int votes[]){
    //count the occurrences of each candidate vote
    for(int i = 0; i < 5; i++){
        votes[i] = 0;
    }

    for(int i = 0; i < arrSize; i++){
        votes[arr[i] - 1]++;
    }
}
```

```
int main(){

    int unsortedArr[maxSize] = {};

    std::string candidates[5] = {
        "Candidate 1",
        "Candidate 2",
        "Candidate 3",
        "Candidate 4",
        "Candidate 5"
    };

    // predetermined seed
    srand(1024);

    // generate random votes with range 1-5
    for(int i = 0; i < maxSize; i++){
        unsortedArr[i] = rand()%5 + 1;
    }

    // print generated unsorted
    std::cout<<"unsortedArr: \n";
    displayArr(unsortedArr);

    // sort using selection sort
    std::cout<<"\nSelection sort: \n";
    selectionSort(unsortedArr, maxSize);
    displayArr(unsortedArr);

    // count votes for each candidate
    int votes[5] = {};
    countVotes(unsortedArr, maxSize, votes);

    //display vote counts
    std::cout<< "\nVote Results: \n";
```

```
    for(int i = 0; i < 5; i++){
        std::cout<< candidates[i] << ": " << votes[i] << "
        votes\n";
    }

    //find the winning candidate
    int winner = 0;

    for(int i = 1; i < 5; i++){
        if(votes[i] > votes[winner]){
            winner = i;
        }
    }

    std::cout << "\nWinner: " << candidates[winner] << ". " <<
    votes[winner] <<" votes\n";

    return 0;
}

void displayArr(int arr[]){
    for(int i = 0; i < maxSize; i++){
        std::cout << arr[i] << " ";
    }
    std::cout << std::endl;
}
```

Table 8-2. Screenshot of Algorithm Code.

Output Console Showing Sorted Array	Manual Count	Count Result of Algorithm
unsortedArr: 3 5 1 5 4 5 5 5 1 3 3 4 4 4 5 1 2 2 3 3 3 5 2 3 1 5 2 5 4 2 5 2 4 3 2 4 2 5 5 3 3 4 3 5 2 5 3 4 2 4 1 3 2 4 5 1 5 3 4 4 3 4 1 1 5 1 3 1 3 5 4 3 2 4 5 4 1 4 1 3 1 5 3 2 1 5 4 5 2 2 1 5 4 4 5 1 5 1 3 5 Selection sort: 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2 2 2 2 2 2 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5	17	17
	15	15
	21	21
	21	21
	26	26

Vote Results: Candidate 1: 17 votes Candidate 2: 15 votes Candidate 3: 21 votes Candidate 4: 21 votes Candidate 5: 26 votes Winner: Candidate 5. 26 votes		

Table 8-3. Output Testing.

Justify why you chose to use this sorting algorithm.

Insertion sort is chosen because the problem states that the range of votes only covers 1 to 5 vote choices for the five candidates. So, the step needed to make the counting of votes possible is to use this algorithm to arrange the votes by their value. This makes sure that the vote counting algorithm can correctly count the instances of the vote numbers. The sorting part is done to group these vote choices together before the counting algorithm does its counting and comparisons to determine the winner.

Question: Was your developed vote counting algorithm effective? Why or why not?

The vote counting algorithm was effective in counting the values sorted by the insertion sorting algorithm used previously to help the main function in finding the winning candidate by the number of votes. It uses two loops: One for counting the occurrences for each candidate vote, and the other for counting and placing the votes to the correct areas. To make sure it counts consistently, it resets the elements to zero. The second loop moves the counted instances to the right place to prevent cutting off the element at the fourth instead of the fifth when printing the results.

Conclusion

This activity has demonstrated the concepts of the three sorting algorithms, which are bubble, insertion, and selection sort. Each algorithm has their own implementations in both linked lists and arrays, which are tested in the procedures. These algorithms present their own unique behaviors and time complexities.

The procedures shown mostly use the array implementations for each algorithm to perform the sorting operations for an unsorted array. Inspecting the code, the three algorithms use a nested loop to perform the searching and swapping, which have an average time complexity of $O(N^2)$. The selection sort may seem to have only one loop, which means it is O(N) but it actually is not because it uses a helper function that has another for loop.

The supplementary activity applies these concepts learned from the procedures to a vote counter scenario using the provided sorting algorithms. However, the problem requires a counting algorithm to actually count the votes and determine the winner, so the counting algorithm was implemented through two loops to count the vote instances and place the counted votes to the correct elements.

Overall, this activity has successfully demonstrated the concepts of sorting algorithms and applied the basic algorithms to working array implementations. The procedure answered during the class visualized the process of each algorithm and the behavior when sorting the unsorted arrays. It also applied the use of randomization to generate pseudorandom values with seeds.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Google Gemini**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Explanation and clarification of concepts**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **To explain and clarify the concepts of the three sorting algorithms, and illustrate their behaviors.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.